NEXUS · MULTI-MODEL INFERENCE ENGINE

The Nexus Guide

Everything in one place

How Nexus works, setups for the hardware you have, every launcher setting, every server flag and environment variable, every HTTP endpoint, the log, and what to do when something goes wrong.

FREE · LOCAL · AGPLv3 · github.com/sjnettifee/nexus

Everything about Nexus in one place: how it works, how to set it up for the hardware you have, every launcher setting, every server flag and environment variable, every HTTP endpoint, the log, and what to do when something goes wrong.

The topic pages go deeper on single subjects and are linked from each section: Launcher · Graphics cards and speed · Maximize GPU · Conversation memory · Documents in chat · Access and API keys · Engine / API · Quant Builder

Contents

1. How Nexus works
2. What's in the folder
3. Choosing a model
4. Setups for common hardware
5. The launcher at a glance
6. Graphics cards and speed
7. Conversation memory
8. The web chat
9. Network and access
10. Several computers, one model
11. Server command line
12. Environment variables
13. HTTP API reference
14. Using the engine from C++
15. Files Nexus writes
16. Reading the log
17. Troubleshooting
18. Glossary

1. How Nexus works

A language model is a stack of **layers** (64 for a 27B model, for example). Every token passes through all of them in order. Nexus decides where each layer lives and where each piece of work happens.

Where a layer lives. Each layer is in one of three places:

- **On a graphics card.** You choose how many layers each card holds.
- **In RAM.** Every layer that isn't on a card. The CPU computes it.
- **On the drive, briefly.** Only with Maximize GPU or oversized-prompt protection, and only while a prompt is read. Nothing is ever read from the drive while writing.

Two jobs, two strategies.

	Reading a prompt (prefill)	Writing the reply (decode)
Work	Thousands of tokens at once	One token at a time
Limited by	Arithmetic	How fast the weights can be read from memory
Nexus does	Streams **every** layer through **one** graphics card, the prompt card, even layers that live in RAM or on other cards	Computes each layer **where it lives**: card 1, card 2, … or the CPU

Streaming is what lets a 3 GB card read prompts for a 15 GB model: the card only ever needs room for a layer or two, and thousands of tokens share each trip across the PCIe bus. While writing, one token per trip wouldn't be worth it, so layers stay put.

Memory, not history. The model's working memory of a conversation (the KV cache) stays in RAM between messages. A new message only adds its own tokens; the history is never read again. You can save that memory to disk, cut pieces out of it, archive them and put them back.

Several models, several computers. Each model runs in its own server process with its own port and settings. One model's layers can also be split across computers on your network.

2. What's in the folder

A release is one folder. Keep it together; it runs from anywhere, including a USB stick.

File or folder	What it is
`nexus_launcher` (`.exe`)	The window you use. Starts and stops servers.
`server/nexus_server` (`.exe`)	The dispatcher. Checks your CPU and starts the best of the programs below. Works on any x86 CPU. The launcher finds it next to itself or in `server/` .
`server/nexus_server_v2` , `_v3` , `_v4` + `libnexus_v2/v3/v4`	The engine, built for SSE4.2 (any x86 CPU since 2009), AVX2/FMA (2013+) and AVX-512. A missing tier is fine.
`web/`	The web chat's document readers (`web/vendor/`). Must sit next to the server, or in the folder above it.
`models/`	Put `.gguf` and `.nxm` models here. Created on first run.
`loras/`	LoRA adapters (`.gguf`) to list in the launcher.
`nexus-data/`	Settings, chats, sessions, archives. See Files Nexus writes.
`nexus_harden_tool/`	The Code Hardener companion app (Python).
`nexus_quant_builder/`	The Quant Builder: Hugging Face models to `.nxm` (Python + numpy).
`docs/`	These guides.

Requirements

- Linux or Windows, x86-64. (macOS builds from source; the graphics card features need Vulkan.)
- RAM: roughly the model file's size, plus the conversation memory, plus 1–2 GB. A 4-bit 27B model is about 15 GB.
- For graphics cards: any card with a Vulkan driver. NVIDIA, AMD (including old GCN cards such as the R9 290/390 and HD 7000) and Intel all work. On Linux, AMD cards use Mesa's RADV driver.
- **Maximize GPU** and **oversized-prompt protection** are Linux only.

---

3. Choosing a model

Formats

- **GGUF** (`.gguf`), including split files (`-00001-of-00003.gguf`). Common types: Q2_K, Q3_K, Q4_K, Q5_K, Q6_K, Q4_0, Q5_0, Q8_0, F16, BF16. The IQ types (IQ2_XS, IQ4_XS and so on), Q4_1 and Q5_1 aren't supported yet; the log shows `[nexus] UNSUPPORTED MODEL FORMAT` with the formats it found, instead of crashing.
- **Nexus native** (`.nxm` single file, `.nxmodel` folder), made with the Quant Builder straight from Hugging Face safetensors. NQ4 / NQ5 / NQ6 / NQ8, one scale per 64 values.

Families. Nexus knows the common architectures: Llama, Mistral, Qwen (2, 2.5, 3, 3.5, 3.6, including the hybrid Qwen3.5/3.6 and MoE versions), Gemma (2, 3, 4), Phi, OLMoE and others. The launcher shows the architecture and the detected chat format when you pick a model.

Size. The model must fit in RAM plus whatever your cards hold. Rough sizes at 4-bit: 7–9B ≈ 5 GB, 14B ≈ 9 GB, 27B ≈ 15–16 GB, 32B ≈ 19 GB. Then add the conversation memory, which depends on the model and grows with the conversation (the Memory tab in the web chat shows the cost per token).

Images and audio. Vision models need their image encoder (an `mmproj` GGUF, or built into an `.nxm`); audio needs a Whisper-style encoder. Load them with `--vision-encoder` / `--audio-encoder` , or at runtime through the API.

---

4. Setups for common hardware

Start from the closest one; the launcher's automatic choices usually land here anyway.

CPU only

Run on graphics card(s) → Graphics card 1: **CPU only**. Set **CPU threads** to your physical core count (not threads). Prompts are read on the CPU too. Try *CPU: 8-bit output math* on **Auto**: it measures whether it helps on your CPU.

One small graphics card (model doesn't fit on it)

The most common setup, and the one Nexus was built for.

- Graphics card 1: your card, **Layers kept on card 1**: 0, or a few if there's room.
- *Prompt speed-up*: **Auto** (it turns on). The card reads every prompt; the CPU writes the reply.
- If the card has a few GB free, **Maximize GPU** on card 1 puts as many layers as fit on it for writing too (Linux).

The model fits on the card

Card 1: all layers (the launcher suggests it). The prompt speed-up turns itself off, since there's nothing to stream: the card does everything.

Two or more cards

- Card 1: the card with the most computing power **and** the best PCIe link. Make it the prompt card.
- Card 2, 3, ...: helper cards. Set them to **Maximize GPU** (or a number). They hold layers for writing; their PCIe link barely matters.
- Leave **8-bit prompt math** on **Auto** on every card.
- If prompts got slower after adding a card, read Slots and PCIe lanes.

Very long prompts on limited RAM

A 100,000-token prompt on a 27B needs several GB of conversation memory on top of the model. On Linux:

- **Maximize GPU** on the prompt card: those layers have no copy in RAM, freeing that much.
- **Oversized-prompt protection** (*Prompt speed-up* card): when free RAM falls under your number while reading, layers leave RAM instead of the system swapping.
- Conversation memory → *A set number of tokens* or *What fits in memory* if you need past the model's normal maximum.

Details: MAXIMIZE_GPU.md.

Two computers

Main computer: the one with the better graphics card. Donor: anything with spare RAM. See Several computers, one model.

Example: the development machine

25 GB DDR4-2200, GTX 1060 3 GB in slot 1, R9 290 4 GB in slot 2, Qwen 3.6 27B at 4 bits (64 layers):

Setting	Value
Graphics card 1	GTX 1060, Maximize GPU, 8-bit prompt math Auto (on: Pascal has the hardware)
Graphics card 2	R9 290, Maximize GPU, 8-bit prompt math Auto (off: measured slower)
Prompt speed-up	On, card: GTX 1060
Split long prompt steps	Auto (the prompt card isn't AMD, so off)
Card memory always kept free	768 MB
MTP	Off (most layers run on the CPU, so drafts don't pay off here)

Result: an 11,400-token prompt in about 270 s, over 1 token/s writing, and the desktop stays responsive. With the GTX 1060 alone in the slot (x16 instead of x8), the same prompt took 251 s.

---

5. The launcher at a glance

Three tabs: **Setup** (one card per topic), **Log** and **Donor**. A slot strip at the top runs several models side by side, each its own server. Start/Stop and *Open chat in browser* sit top right. Nothing is written to disk until **Save current settings**. Every setting with **?** has a note.

Card	What it sets
Model	Model file, folders to scan, profiles (named snapshots of every setting for one model).
This computer	Hardware found, and whether each card is usable.
Run on graphics card(s)	Layers per card (number or Maximize), 8-bit prompt math per card, CPU threads, CPU 8-bit output math.
Prompt speed-up	On/off/Auto, prompt card, attention path, split long prompt steps, working space, oversized-prompt protection.
Share with other computers	Donors, split, prompt reading across computers.
Model behaviour	Chat format, RoPE layout, norm convention, thinking. Saved per model file.
Conversation memory	Length limit and auto-trim.
Sampling	Temperature, top-p, top-k, min-p, tail-free, penalties, max tokens.
Images	What vision models see and how big uploads are.
LoRA adapter	One adapter and its strength.
Speculative decoding (MTP)	Draft heads on/off, tokens per step, CPU or card.
Saved state at startup	A `.kvc` warm state loaded before serving.
Network	Port, who can connect, API key, which web pages may use it.
Speed and accuracy	Q8_0 output layer, 8-bit K-quant math (both on by default).
Diagnostics	Quick self-test, token-by-token prompts, profile, log detail.
Advanced	Memory safety buffer, extra server options, extra `NEXUS_` variables.

Every card in detail: LAUNCHER_GUIDE.md.

6. Graphics cards and speed

The full page is GRAPHICS_CARDS_AND_SPEED.md; this is the summary.

Every speed setting

Setting	Choices (default first)	Effect
How card N holds its layers	**A set number** / Maximize GPU	Maximize measures the card's real free memory and fills it.
8-bit prompt math on card N	**Auto** / On / Off	Rounds activations (never weights) to 8 bits for the card's integer units. ~0.5% per matrix. Auto keeps it only where it's faster.
CPU: 8-bit output math	**Off** / Auto / On	Same idea on the CPU while writing NQ models, ~1% per step. Auto keeps it only if ≥10% faster.
Split long prompt steps	**Auto (AMD only)** / On / Off	Keeps each step under 3 s for AMD's Linux driver time limit. Never below 256 tokens.
Prompt working space on the card	**Fit to what's really free** / By the card's size	How much of the prompt card the prompt speed-up may fill.
Card memory always kept free	**768** MB	Left free on every Maximized card.
Prompt card: memory for reading a prompt	**auto** / MB	With Maximize on the prompt card: room a prompt gets.
Layers swapped through the card at once	empty	More than 1 can help on cards with lots of memory.
Attention path	**Auto** / Stream / Batch	Short prompts batch.
Store the output layer as Q8_0	**On** / Off	Faster; tiny rounding change.
8-bit math for K-quant matrices	**On** / Off	GGUF K-quants while writing; the llama.cpp trade.
MTP	**Off** / On, drafts per step, CPU / card	Lossless; helps when most layers are on cards.

What helps most

1. **Put layers on cards** for writing speed. Even an old helper card beats the CPU: its memory is several times faster than system RAM.
2. **Prompt card on a full-speed slot.** Every prompt sends the whole model over its PCIe link.
3. **Threads = physical cores.** More threads than cores slows writing.
4. **Leave 8-bit on Auto** on every card and try Auto on the CPU.
5. **MTP** only when most layers are on cards. Measure with it on and off.

Older AMD cards

GCN 1 and 2 cards (R9 290/390, HD 7000) work and make good helper cards. Auto 8-bit measures them and picks the regular kernels (they only fake 8-bit). They run on the compute queue, which has a 60 s driver limit instead of 10 s, and *Split long prompt steps* keeps each step short.

---

7. Conversation memory

Full page: CONVERSATION_MEMORY.md.

- **Limit**: *the model's maximum* (default), *a set number of tokens* (exact, even above the maximum), or *what fits in memory* (no limit).
- **Auto-trim** (on by default): when full, drop the oldest whole messages down to ~75% of the limit and keep going. The system prompt and the message being answered are never dropped.
- **Forget** any message: removed from memory, and everything after it is shifted back with its positions recalculated, as if it had never been there.
- **Archive** a message as A1, A2, … (saved to disk, then forgotten), and **Restore** it later into any chat with the same model, with no re-reading.
- **Recurrent models** (Qwen3.5/3.6, Qwen3-Next): their linear-attention layers keep a running summary that can't drop single tokens. Forget is exact in the attention layers; the summary keeps a faint trace.

Which "save" is which?

	What it keeps	Use it for
Session (Sessions tab → Save)	The whole conversation memory and its messages	Pick a chat up later exactly where it was
Warm state (Sessions tab, or `--start-state`)	Memory only, as a starting point	A long system prompt or reference you want every new chat to start with
Checkpoint	A snapshot of this exact point	Before trying something risky in the same chat
Archive (A1, A2, …)	One part of a conversation	Take a document or a code revision out of memory and put it back when needed
Export (.html, .md)	The text on screen	Sharing or posting a chat; can't be continued

8. The web chat

Open chat in browser opens the chat built into the server at `http://<address>:<port>/`. It works from other computers too (see Network and access).

Writing

- **Send** (Enter; Shift+Enter for a new line). While a reply is being written, the same button stops it.
- **Send + Think** asks a reasoning model to think first, for this message only. Its thinking is shown folded under *Thinking*.
- The status bar shows *in memory: N / limit tok* (the engine's own count) and, during a long prompt, percent, speed and time left.

Attaching

- Drag files in, or use the attach button. Images go to vision models.
- Documents are read as text in the browser: PDF (with OCR for scanned pages), Word, spreadsheets, slides, e-books, RTF, HTML. A chip shows progress; Send waits until every chip shows its file name. See DOCUMENTS_IN_CHAT.md.
- Drag a folder in, or use **DIR**: every text file and document inside is attached under its path. Build output, dependencies, binaries and lock files are skipped, and it asks first if the folder won't fit.

Each message has **Copy** (Markdown, code blocks kept) and ⋯: Forget, Archive, Select (several at once), Restore.

Sidebar

Tab	What's there
Sessions	Save / Load sessions (and extra folders to save in), Warm States, Checkpoints, Export as .html or .md
Memory	Usage bar, limit and auto-trim, Saved memories with Restore, Rename, Copy text, Delete
Settings	Model info; Documents (OCR on/off, OCR language); live sampler, with *Save as defaults*; display options; a system prompt for the next message and *warm* it

Top bar: *New Chat*, *Connect to session* (reattach to a reply the server is still writing after you closed or reloaded the page), *Clear*.

A reply keeps going on the server even if the page closes. Reopen the chat and use *Connect to session*.

9. Network and access

Full page: ACCESS_AND_API_KEYS.md.

Want	Do
Use it on this computer only	Nothing. Default.
Use it from other devices at home	Launcher → Network → *Who can connect*: *Anyone on my network*. Allow it through the firewall on Windows.
Make other computers prove they're allowed	**Make a key**. The web chat asks for it once per browser; programs send `Authorization: Bearer <key>`.
Your own web app on your network	Works. Pages from this computer, private addresses, computer names and `.local` names are allowed.
Your own web app on the internet	Add its address under *Also allow web pages from* (`--allow-origin`).
Reach it across the internet	Put it behind HTTPS (a reverse proxy) or use Tailscale. The key travels in plain text over plain HTTP.

Internet websites can't use your models by default, even though your browser can reach your server. Every refusal is in the log as `[access] refused …` with the reason.

10. Several computers, one model

One model's layers can be split between a main computer and one or more donors. Each computer holds and computes its share; only the activations (a few KB per token) cross the network, so a wired connection is best.

In the launcher: on the donor, the **Donor** tab (port, passphrase, model file if it has one, threads, card) → **Start donor**. On the main computer, *Share with other computers* → add each donor's address, port and passphrase → check the **Layer plan** → Start. Details: LAUNCHER_GUIDE.md.

From the command line:

```
# donor
nexus_server --dist-worker --port 9876 --dist-passphrase <word> \
             [--dist-local-model /path/model.gguf] [--gpu-layers N]

# main computer: layers 0-19 here, 20-39 on the donor
nexus_server --model /path/model.gguf --port 8080 \
             --dist-layer-start 0 --dist-layer-end 20 \
             --dist-peer donor-host:9876 --dist-peer-layers 20:40 \
             --dist-passphrase <word> [--dist-peer-local-load] [--dist-required]
```

Prompts across computers: in pieces (`NEXUS_DIST_PREFILL_CHUNK`, default 1024 tokens), so both computers work at once, or the whole prompt on the main computer (*hybrid prefill*, `NEXUS_HYBRID_PREFILL=1`, from `NEXUS_HYBRID_PREFILL_MIN_TOKENS`, default 2048), which helps when the main computer's card is much faster.

Not yet across computers: conversation memory edits (`/api/kv/*`) and images in `extend_turn`. Sessions save and load on both computers.

The passphrase authenticates the connection; it is never sent over the network itself. Treat the link as unencrypted and keep donors on a network you trust.

11. Server command line

`nexus_server --help` prints the list. The launcher passes all of these for you; you only need them to run headless.

Basics

Flag	Meaning
`--model <path>`	Model file (`.gguf` , `.nxm` , `.nxmodel`)
`--port <n>`	Port (default 8080)
`--host <addr>`	Listen address. `127.0.0.1` (default) = this computer only; `0.0.0.0` = network
`--data <dir>`	Where chats, sessions, states and archives go (default `nexus-data`)
`--threads <n>`	CPU threads
`--config <path>`	Read settings from a config file
`--roster-file <path>`	Model roster for the web chat's model picker (the launcher writes it)
`--trace-level <0-3>`	Log detail
`--list-devices`	List graphics cards and exit
`--probe`	Print a hardware report (JSON) and exit
`--help`	The flag list

A `nexus.json` in the working folder supplies `model_path` , `host` , `port` and `threads` for anything not on the command line. Flags always win.

Chat format

Flag	Meaning
`--chat-format <fmt>`	`ChatML` , `Qwen3` , `Mistral` , `Mistral V3` , `Llama-3` , `DeepSeek` , `DeepSeek-R1` , `DeepSeek-Coder` , `Phi-3` , `Phi-4` , `Gemma` , `Gemma4` , `Zephyr` , `Alpaca` , `Granite` , `OLMoE` , `Custom` , `Raw`
`--chat-custom-pre-prompt` / `-pre-system` / `-end-system` / `-pre-user` / `-end-user` / `-pre-model` / `-end-model` <s>	The pieces of a `Custom` format

Graphics cards

Flag	Meaning
`--gpu-layers <n>`	Layers on the first card
`--gpu-device <n | name:<card name>[#k]>`	Which card. Names are safer than numbers

Flag	Meaning
`--gpu2-layers <n | auto>`, `--gpu2-device`	Second card; `auto` = as many as fit
`--gpuN-layers`, `--gpuN-device` (N ≥ 3)	Further cards

Prompt speed-up

Flag	Meaning
`--turbo-prefill`	Batched prompt reading
`--turbo-gpu-stream`	Stream layers through the card while reading (the prompt speed-up)
`--turbo-vram-budget-mb <n>`	Cap the card memory it uses
`--prefill-token-by-token`	Diagnostic: one token at a time

Maximize GPU and protection (Linux)

Flag	Meaning
`--park-layers <N | auto>`	Maximize on the prompt card: layers live there for writing with no RAM copy
`--park-below-mb <MB>`	Oversized-prompt protection threshold
`--park-prefill-free-mb <MB | auto>`	Room the prompt speed-up gets on the prompt card
`--park-keep-mb <MB>`	Kept free on every Maximized card (default 768)
`--park-min-tokens <N>`	Prompts shorter than N never move layers (default 0 = off)

Conversation memory

Flag	Meaning
`--ctx-limit <n | auto | max>`	Conversation length limit (default `max`)
`--auto-trim <on | off>`	Drop oldest messages when full (default on)
`--start-state <path>`	Load a saved warm state before serving

Sampling defaults (per-request values override them)

`--temperature`, `--top-p`, `--top-k`, `--min-p`, `--tfs-z`, `--rep-penalty`, `--repeat-last-n`, `--freq-penalty`, `--presence-penalty`, `--max-tokens`.

MTP

Flag	Meaning
`--mtp <on | off>`	Use the model's draft heads
`--mtp-n-predict <n>`	Tokens drafted per step

Flag	Meaning
`--mtp-gpu <on | off>`	Draft on the prompt card

Adapters and encoders

Flag	Meaning
`--lora <path>`, `--lora-scale <f>`	A LoRA adapter (GGUF) and its strength
`--vision-encoder <path>`	Image encoder (`mmproj` GGUF)
`--audio-encoder <path>`	Audio encoder (Whisper GGUF)

Second model in the same server

`--model2 <path>`, `--model2-gpu-layers <n>` (-1 = all), `--model2-gpu-device <n>`, `--model2-threads <n>`, `--model2-chat-format <fmt>`. Address it with `"model":"2"` in API calls. (The launcher runs each model as its own server instead.)

Access

Flag	Meaning
`--api-key <key>`	Require a key from other computers (or `NEXUS_API_KEY`)
`--cors <lan | local | any>`	Which web pages may use the API (default `lan`)
`--allow-origin <url>`	Also allow this page address; repeat or comma-separate
`--enable-web-tools`	Turn on `/api/search*` (off by default)

Several computers

`--dist-worker`, `--dist-local-model <path>`, `--dist-passphrase <word>`, `--dist-layer-start <n>`, `--dist-layer-end <n>`, `--dist-peer <host:port>`, `--dist-peer-layers <start:end>`, `--dist-peer-local-load`, `--dist-required`. See section 10.

Environment variables

Settings without their own flag. In the launcher, put them under *Advanced → Extra environment variables* (one `NAME=value` per line; only `NEXUS_` names are accepted). The launcher already sets the ones its controls cover; this table says which control.

Graphics cards and prompts

Variable	Values	Effect	Launcher control
`NEXUS_MM_INT8_CARDS`	`name=1;name=0`	8-bit prompt math per card, by card name. A card not listed is on Auto	8-bit prompt math on card N
`NEXUS_MM_INT8`	`1` / `0`	Force 8-bit prompt math on or off for every card	—
`NEXUS_MM_NOINT8`	`1`	Never use 8-bit prompt math	—
`NEXUS_PREFILL_TILE_MAX_S`	seconds; `0` = off	Split a prompt step that takes longer than this. Unset = AMD only, 3 s	Split long prompt steps
`NEXUS_PREFILL_TILE_MIN`	tokens (256)	Smallest step size when splitting	—
`NEXUS_TURBO_BUDGET_REAL`	`0`	Size the prompt working space by the card's total memory instead of what's really free	Prompt working space
`NEXUS_TURBO_GPU_LAYERS`	n	Layers swapped through the card at once	Layers swapped through the card at once
`NEXUS_ATTN_PATH`	`stream` / `batch`	Force the attention path	Attention path
`NEXUS_VK_GFX_QUEUE`	`1`	AMD: use the graphics queue instead of compute (old behaviour)	—
`NEXUS_EVICT_RECENT`	n (24)	How many recently used weights the card's cache never evicts	—
`NEXUS_VULKAN_ALLOW_SOFTWARE`	`1`	Allow a software Vulkan renderer (llvmpipe). Slow; testing only	—
`NEXUS_PREFILL_PROFILE`	`1`	Print a prompt-speed profile	Print prompt-speed profile

Writing (output)

Variable	Values	Effect	Launcher control
`NEXUS_CPU_INT8`	`1` / `auto` / unset	CPU 8-bit output math for NQ models	CPU: 8-bit output math
`NEXUS_LMHEAD_Q8`	`0`	Keep the output layer exact instead of Q8_0	Speed and accuracy
`NEXUS_DECODE_INT8`	`0`	Exact math for K-quant matrices while writing	Speed and accuracy

Memory and safety

Variable	Values	Effect	Launcher control
`NEXUS_MEM_GUARD`	`0`	Turn the memory safety buffer off	Memory safety buffer
`NEXUS_MEM_BUFFER_MB`	MB (1024)	RAM the safety buffer keeps free	Memory to keep free

Model behaviour

Variable	Values	Effect	Launcher control
`NEXUS_ROPE_LAYOUT`	`interleaved` / `half_split`	Override RoPE layout	RoPE layout
`NEXUS_NORM_CONVENTION`	`direct` / `zero_centered`	Override the norm convention	Norm convention
`NEXUS_ENABLE_THINKING`	`1` / `0`	Thinking on/off for models that have the switch	Thinking
`NEXUS_PRESERVE_THINKING`	`1` / `0`	Keep earlier turns' thinking in the prompt	Keep earlier thinking

Images

Variable	Effect	Launcher control
`NEXUS_QWEN_NATIVE_IMAGE_SIZE`	Image size the model sees (px)	Image size the model sees
`NEXUS_WEB_IMAGE_MAX_EDGE`, `NEXUS_WEB_IMAGE_MAX_PIXELS`	Largest uploaded edge / area; larger uploads are shrunk	Largest uploaded edge / area
`NEXUS_VISION_MODE`	`single` / `tiles`	Large images
`NEXUS_VISION_TILE_SIZE`, `_TILE_STRIDE`, `_MAX_TILES`	Tiling	Tile size, step, most tiles
`NEXUS_VISION_GPU`	`1` / `0`: image encoder on the card	Run the image part on the graphics card

Several computers

Variable	Effect	Launcher control
`NEXUS_DIST_PREFILL_CHUNK`	Prompt piece size across computers (1024; 0 = whole prompt, layer by layer)	Reading long prompts
`NEXUS_HYBRID_PREFILL`	`1` = read the whole prompt on this computer	Read long prompts entirely on THIS computer
`NEXUS_HYBRID_PREFILL_MIN_TOKENS`	From this prompt length (2048)	the token count next to it

Server

Variable	Effect
`NEXUS_API_KEY`	API key (instead of `--api-key`; stays out of the process list)
`NEXUS_WEB_DIR`	Folder holding `vendor/` for the web chat's document readers
`NEXUS_TRACE`	Log detail 0–3 (same as `--trace-level`)

Other `NEXUS_` names in the source are developer diagnostics and may change between versions.

---

13. HTTP API reference

Plain HTTP and JSON; replies stream as Server-Sent Events. Every call that touches a model takes `"model":"1"` (default) or `"2"`. Requests from other computers need the API key when one is set. Explanations and examples: ENGINE_API_GUIDE.md and ACCESS_AND_API_KEYS.md.

A typical conversation

```
POST /api/chat                {"messages":[{"role":"user","content":"Read this: ..."}]}
  ← {"conv_id":"c1"}  {"progress":{...}} ...  {"token":"..","gen":1,"context_tokens":40211}
...  {"final":true,...}  [DONE]
POST /api/session/extend_turn {"conv_id":"c1","text":"Now summarize section 3"}
POST /api/continue            {"conv_id":"c1"}
  ← tokens ...
GET  /api/kv/turns            → messages in memory with their token ranges
POST /api/kv/archive          {"start":..,"end":..,"ctx":..,"label":"the manual"}   → A1,
memory freed
POST /api/kv/archive/restore  {"id":"A1","ctx":..}                                  → back,
no re-reading
```

Stream events

Event	Meaning
`{"conv_id":"..."}`	First event. Send it back to continue
`{"progress":{"pct","tokens","tps","eta"}}`	Prompt reading
`{"token":"...","gen":N,"context_tokens":M}`	Text
`{"notice":"..."}`	Show the user (memory safety stop, memory full)
`{"final":true,"gen":N,"context_tokens":M}`	Exact totals
`{"nojob":true}`	`/api/continue/resume` : the server no longer holds that reply
`[DONE]`	End

Endpoints

Generating

Endpoint	Use
`POST /api/chat`	Messages in, streamed reply out. `conv_id` reuses memory; `turn_id` makes retries safe; `think` , `images` , sampling fields, `mtp_*`
`POST /api/session/extend_turn`	Add one user message (with images) to memory
`POST /api/session/extend`	Add raw text, no chat formatting

Endpoint	Use
`POST /api/inject`	Add a system/tool-result block, ready for the model to answer
`POST /api/continue`	Generate from the current memory, no prompt
`POST /api/continue/resume`	Reattach to a reply after a dropped connection
`GET /api/progress`	Prompt reading progress
`POST /api/stop`	Stop the reply
`POST /api/reset`	Clear the conversation

Memory

Endpoint	Use
`GET /api/context`	Tokens in memory, limit, remaining
`GET /api/session/tokens`	The raw token list
`GET /api/kv/turns`	Messages in memory with token ranges
`POST /api/kv/remove`	Forget a range
`POST /api/kv/archive`	Save a range as A, then forget it
`GET /api/kv/archive/list`	Archives
`POST /api/kv/archive/restore` / `rename` / `delete`	Manage archives
`POST /api/kv/settings`	Limit and auto-trim while running
`POST /api/session/save` / `load` / `delete`	Whole-conversation sessions
`GET /api/sessions/list`, `POST /api/sessions/list_in`, `POST /api/sessions/meta`	Browse saved sessions
`POST /api/checkpoint/save` / `restore` / `delete`, `GET /api/checkpoint/list`	Checkpoints
`POST /api/start_state/save` / `load` / `delete`, `GET /api/start_state/list`	Warm states
`POST /api/slot/select`, `GET /api/slot/list`, `POST /api/slot/clear`	Several conversations resident at once

Model and tools

Endpoint	Use
`GET /api/info`	Model name, context length, format, model 2
`GET` / `POST /api/config`, `POST /api/config/save`	Server settings
`GET` / `POST /api/turbo`	Prompt speed-up settings at runtime

Endpoint	Use
`POST /api/preload`	Warm up a model
`POST /api/tokenize`, `POST /api/detokenize`	Text ↔ token ids
`GET /api/top_logits?n=k`	The k most likely next tokens
`POST /api/embed`	Embedding vector of a text
`POST /api/lora/load` / `unload` / `scale`, `GET /api/lora/list`	LoRA adapters at runtime
`POST /api/vision/load`, `GET /api/vision/status`, `POST /api/vision/inject`	Image encoder
`POST /api/audio/load`, `GET /api/audio/status`, `POST /api/audio/inject`, `POST /api/audio/encode_wav`	Audio encoder
`POST /api/image/upload`	Upload an image once, reference it by id
`GET /api/roster`	The model roster
`GET /api/search`, `/api/search_detail`, `/api/search_detail_by_url`	Web search for tools (only with `--enable-web-tools`)

Web chat's own storage (only needed if you rebuild the web chat): `/api/projects`, `/api/chats`, `/api/templates`, `/api/folders`, `/api/live/*`, `/api/boot`. Static files: `/` and `/vendor/…`.

---

14. Using the engine from C++

The server is a thin layer over `nexus::Engine` and `GenerateSession`. Link `engine.h` / `engine.cpp` to use Nexus inside your own program with no HTTP at all:

```
#include "api/engine.h"

nexus::Engine engine;
engine.load("model.gguf");
engine.set_gpu_layers(28);

nexus::GenerateConfig cfg;
cfg.max_tokens = 512;
engine.extend_context(engine.format_user_turn("Hello!"), /*add_bos=*/true);
std::cout << engine.generate(cfg).text;
```

Streaming sessions, context management, persistence, LoRA, vision, audio and the layer-streaming primitives are covered in ENGINE_API_GUIDE.md, section 4.

15. Files Nexus writes

Everything lives next to the launcher, in `nexus-data/` (the servers run with the launcher's folder as their working folder).

Path	What
`nexus-data/nexus_launcher.json`	Launcher settings, profiles, per-model settings (written on *Save current settings*)
`nexus-data/nexus-roster.json`	The models the launcher runs, for the web chat
`nexus-data/sessions/`	Saved sessions: `<name>.kv` (memory) + `<name>.tokens`
`nexus-data/states/`	Warm states (`.kvc`)
`nexus-data/checkpoints/`	Checkpoints (`.kvc` + `.json`)
`nexus-data/kv_archive/`	Archives: `A1.nxctx` (memory) + `A1.json` (name, model, date, text)
`nexus-data/projects/` , `chats/` , `templates/`	The web chat's projects, chat history and prompt templates
`nexus.json`	Server defaults (sampler *Save as defaults* writes here)

Session files can be large: a conversation's memory is its token count times the model's bytes per token (shown in the Memory tab). Sessions only load into the model they were saved from.

16. Reading the log

The **Log** tab (or the server's console) says what happened in plain words. The lines worth knowing:

Line starts with	Meaning
`[load] NN% …`	Loading. *Moving layers onto graphics card N* = filling a helper card
`[nexus] UNSUPPORTED MODEL FORMAT …`	The file uses a weight format Nexus can't run yet; the line lists which
`[nexus] graphics card N (...): K layers ...`	How many layers a helper card got, and its free memory after
`[gpu] "<name>" is graphics card N on this computer`	Which card a name resolved to
`[vulkan] 8-bit prompt math: ...`	What each card chose, and why (measured times on Auto)
`[cpu] 8-bit output math (Auto): ...`	What the CPU measured and picked
`[progress] reading prompt: NN% (layer i of n, … tok/s, …)`	Prompt progress, every 2 s
`[progress] moving layers onto the card ...`	Maximize GPU: layers going back after a prompt
`[turbo:auto-budget] ... source=real-free`	The prompt speed-up's room on the card
`[prefill] layer N: a ...-token slice took ... s ...`	*Split long prompt steps* kicked in
`[park] ...`	Maximize GPU / protection moving layers
`[nexus] graphics card ... is nearly full while writing ...`	A card gave a layer back to the CPU
`[vulkan] cache full and nothing it can evict ...`	A card is completely full: raise *Card memory always kept free*
`[mtp-time] step=… propose=…ms verifyN(ntok=…)=… ms`	MTP timing per step
`[mtp-diag] … MATCH / miss`	Which drafts were kept
`[ctx] ...`	Conversation limit, auto-trim, memory full
`[access] refused <method> <path> from <addr>: <reason>`	A request was refused, and why
`What probably happened: ...`	Nexus's own explanation of an error (highlighted)

Log detail 2–3 adds per-step timing, mostly useful when reporting a problem.

Troubleshooting

Starting

Symptom	Fix
Server program not found notice	Keep `nexus_server*` and `libnexus_*` next to the launcher.
No model listed	Put the file in `models/`, or *Add folder...*, then *Rescan*.
Port won't bind	Another program uses it (8080 is popular on Windows). Change the port in Network.
A model won't start with donors on	A donor's address or passphrase is empty. Nexus refuses rather than loading everything locally.
Another device can't connect	*Who can connect* → *Anyone on my network*; Windows: allow both programs in the firewall.
401 / *needs its API key*	The other computer isn't sending the key.

Output

Symptom	Fix
Gibberish right after loading	Model behaviour → try the other RoPE layout or norm convention. The log says which was used.
Odd turn markers or the model talks to itself	Wrong chat format. Pick it by hand in Model behaviour.
Reply stops with *Memory full*	Raise the limit or turn auto-trim on (Conversation memory).
Reply stops with a memory safety message	RAM is nearly gone. Free memory, shorten the conversation, put more layers on cards, or use Maximize / protection.
The model "feels" something you made it forget	Recurrent models (Qwen3.5/3.6) keep a faint trace in their linear-attention layers. Start a new chat for a clean slate.

Speed

Symptom	Fix
Prompts slower after adding a card	PCIe split to x8. Move the helper card to a chipset slot (often x4 is fine for it) or use a riser.
Prompts slow on an NVIDIA card	Its 8-bit prompt math isn't Off? Check the `[vulkan] 8-bit prompt math` line.
Prompts slow on an old AMD card	Expected with 8-bit forced On; set it to Auto.
Writing slow	More layers on cards; threads = physical cores; try CPU 8-bit output on Auto; MTP off on CPU-heavy splits.
MTP slower than without	Most layers are on the CPU. Try 2 drafts, or turn it off.
The computer freezes during long prompts	Memory safety buffer on; on Linux, Maximize GPU or oversized-prompt protection.

Graphics cards

Symptom	Fix
A helper card stays empty	Look for its `[nexus] graphics card N` line at load. If it says it doesn't fit, lower its layers or use Maximize.
Out-of-memory messages	*Card memory always kept free* → 1024; *Prompt working space* → really free. More if the desktop runs on that card.
AMD card resets during a long prompt ("context is lost")	*Split long prompt steps* → On.
A card is missing from the list	*This computer* → *Re-check hardware*; check its Vulkan driver (`vulkaninfo --summary`).
Integrated graphics gives wrong answers	It's off by default for that reason. Use the CPU or a discrete card.

Documents

Symptom	Fix
Readers are missing	`web/` isn't next to the server. Copy it, or set `NEXUS_WEB_DIR` .
A scanned PDF reads slowly	OCR takes a few seconds per page. Turn it off in Settings → Documents if you don't need it.
`.doc` / `.ppt` refused	Save as `.docx` / `.pptx` or PDF first.

When reporting a problem, include the Log tab's contents (**Copy all**) with *Log detail* at 2.

---

18. Glossary

Term	Meaning
Layer	One block of the model. Every token passes through all of them in order.
Prefill	Reading the prompt. Many tokens at once; limited by arithmetic.
Decode	Writing the reply, one token at a time; limited by memory speed.
Prompt card	The graphics card the prompt speed-up streams layers through.
Helper card	Any other card. Holds layers for writing.
KV cache	The model's working memory of the conversation. Grows with every token.
Context / limit	How many tokens are in memory, and how many are allowed.
Quantization	Storing weights in fewer bits (4, 5, 6, 8) to save memory.
8-bit math	Rounding the numbers between layers (not the weights) to 8 bits so integer units can multiply them faster.
MTP	Multi-token prediction: the model's own draft heads guess ahead, and one pass checks the guesses.
RoPE	How the model encodes token positions. Recalculated when part of the memory is removed.
Linear attention / recurrent layers	Layers (in Qwen3.5/3.6, Qwen3-Next) that keep a running summary instead of per-token memory.
Maximize GPU	Fill a card with as many layers as really fit.
Parking	On the prompt card with Maximize: layers that live on the card with no RAM copy and move off only when a prompt needs room.
Donor	A computer lending its CPU, RAM and card to another computer's model.
VRAM	A graphics card's own memory.
PCIe x16 / x8 / x4	How many lanes connect a card to the CPU. The prompt card wants as many as possible.
Vulkan	The graphics card interface Nexus uses on every brand.
Tier (v2 / v3 / v4)	Engine builds for older, newer and newest CPU instructions. The dispatcher picks one.